

CRM-Based Web Call Dialer - System Architecture, Workflow & Technology Documentation

TABLE OF CONTENTS

1. Introduction & Problem Statement

2. What is CRM?

- 2.1 Information Managed by CRM
- 2.2 How CRM Helps the Sales Team
- 2.3 CRM and Web Dialer Integration

3. User Roles & Responsibilities

- 3.1 Admin
- 3.2 Manager
- 3.3 Sales Executive

4. Main Features

5. High-Level System Architecture

- 5.1 Components of the System

6. Web Call Dialer Architecture

- 6.1 Why Use a Calling Service
- 6.2 Dialer Component Flow
- 6.3 Responsibilities of Backend and Calling Service

7. Complete Call Workflow

- 7.1 Step-by-Step Call Process
- 7.2 Successful Call
- 7.3 Failed Call

8. Call Status, Duration & Call History

- 8.1 Call Status
- 8.2 Call Duration
- 8.3 Call History

9. Database / ER Diagram

- 9.1 Users Table
- 9.2 Leads Table
- 9.3 Calls Table
- 9.4 FollowUps Table
- 9.5 ER Diagram

10. Basic API Design

11. Authentication & Role-Based Access Control

- 11.1 Authentication
- 11.2 Role-Based Access Control (RBAC)

12. Technology Stack & Reasons for Choosing Each Technology

13. Security

14. Basic Scalability

15. End-to-End Example

16. Conclusion

1. Introduction :

Sales teams in most companies spend a large part of their day calling leads and customers. In many companies, this calling process is disconnected from the CRM (Customer Relationship Management) system. A Sales Executive has to look up a phone number in the CRM, then dial it manually from a mobile phone or a separate desk phone, and then come back to the CRM to write down what happened during the call. This manual process is slow, inconsistent, and easy to get wrong.

The main problems this project tries to solve are:

- Sales Executives waste time switching between the CRM and a phone to make calls.
- There is no proper record of call duration, call status, or call outcome inside the CRM.
- Managers cannot easily see how many calls their team made, or how leads are progressing.
- Follow-ups are often forgotten because there is no reminder system linked to the call.
- Admin has no simple place to manage users, roles and permissions.

The CRM-Based Web Call Dialer solves this by allowing a Sales Executive to click a single "Call" button directly inside the CRM. The system connects the call, tracks the call status and duration automatically, and stores the call history against the correct lead. Managers get a dashboard to monitor their team, and Admin gets full control over users and access rights.

2. What is CRM?

CRM stands for Customer Relationship Management. A CRM system is software that helps an organization manage its interactions and relationships with customers and potential customers.

In a sales organization, a CRM acts as a **centralized system for managing leads, customers, sales activities, communication, and follow-ups.**

Instead of storing customer information in separate spreadsheets, notebooks, or different applications, the CRM keeps the information in one system that authorized users can access according to their roles.

2.1 What information does a CRM manage?

A CRM can maintain information such as:

- Customer name
- Phone number
- Email address
- Company information
- Lead status
- Assigned Sales Executive
- Call history
- Call duration
- Call notes
- Follow-up dates

- Sales activities

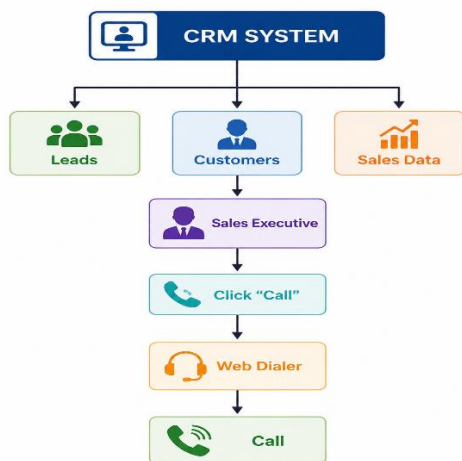
2.2 How CRM helps the sales team

The CRM provides Sales Executives with a centralized view of their assigned leads. They can open a customer's record, view relevant information, contact the customer, update the lead status, add notes, and schedule follow-ups.

Managers can use the CRM to monitor their team's activities and review information such as calls made, call status, call duration, and customer interactions.

2.3 CRM and Web Dialer Integration

The **CRM manages the business and customer information**, while the **web dialer handles the call initiation**.



For the proposed system, the CRM provides the customer/lead information and the interface from which the user initiates the call. The dialer then handles the calling process through the organization's telephony infrastructure.

3. User Roles

The system uses **Role-Based Access Control (RBAC)** to provide different permissions to different users.

There are three primary roles:

1. **Admin**
2. **Manager**
3. **Sales Executive**

Each role performs a different function within the CRM.

3.1 Admin

The **Admin** is responsible for the overall management and configuration of the CRM system.

The Admin has the highest level of access and is responsible for managing users, roles, permissions, system settings, and overall organizational data.

Responsibilities

- Create and manage user accounts.
- Create and manage Managers and Sales Executives.
- Assign roles and permissions.
- Manage system-level settings.
- Manage customers and leads.
- View overall system reports.
- Monitor overall sales activity.

Example

An Admin can create a new Sales Executive account and assign that user to a particular Manager. The Admin can also control what features the user is allowed to access.

3.2 Manager

The **Manager** is responsible for managing the Sales Executives under their team and monitoring their sales activities.

The Manager has access to team-level information but does not have the same system-level privileges as the Admin.

Responsibilities

- View and manage assigned Sales Executives.
- Assign leads to Sales Executives.
- Monitor sales-team activities.
- View call status.
- View call duration.
- View call history.
- Monitor team performance.
- Review lead and customer activity.
- Track follow-ups.

Example

A Manager may receive 100 customer leads and distribute them among 10 Sales Executives. The Manager can then monitor how many calls each Sales Executive has made and review their call history.

3.3 Sales Executive

The **Sales Executive** is the primary user who interacts with customers through the CRM.

The Sales Executive works with the leads assigned to them and uses the web dialer to communicate with customers.

Responsibilities

- View assigned leads.
- View customer information.
- Initiate customer calls through the web dialer.
- View their call history.
- View call status and duration.
- Update lead status.
- Add notes after a call.
- Schedule customer follow-ups.

Example

A Sales Executive opens an assigned lead and sees the customer's name and phone number. The Sales Executive clicks the **Call** button, initiates the call through the web dialer, and after the interaction updates the lead status and adds notes about the conversation.

4. Main Features

4.1 Common Features (All Roles)

- Secure login with username/email and password.
- Role-based dashboard that shows only the information relevant to that user.
- View lead and customer details.

4.2 Sales Executive Features

- View list of leads assigned to them.
- Click a "Call" button to call a customer directly from the CRM.
- See real-time call status (for example: Dialing, Ringing, Connected, Completed, Failed).
- View call duration automatically after the call ends.
- Add notes for a lead after a call.
- Update lead status (for example: New, Contacted, Interested, Not Interested, Converted).
- Set follow-up date and reminder for a lead.
- View their own call history.

4.3 Manager Features

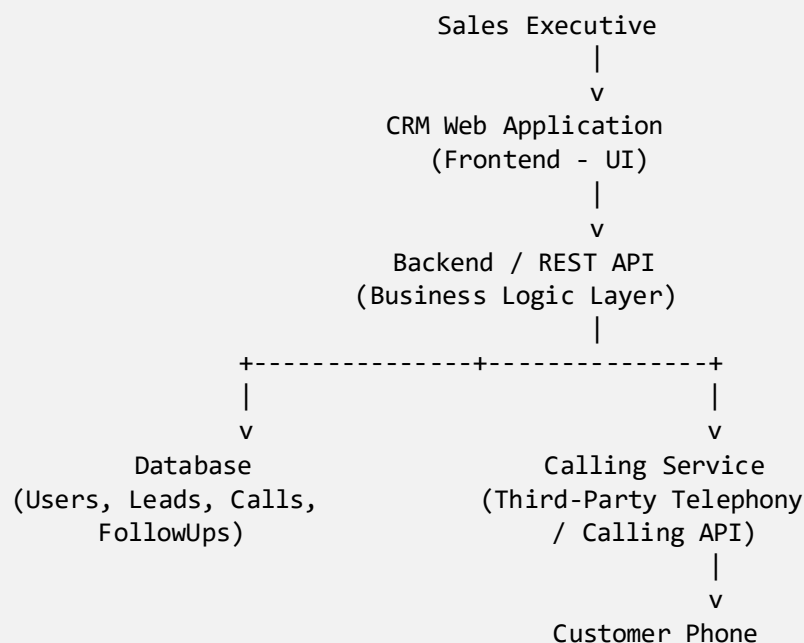
- View all leads and calls handled by their team.
- View reports such as total calls made, average call duration, and missed follow-ups.
- Reassign a lead from one Sales Executive to another, if needed.
- Track team performance over a day, week, or month.

4.4 Admin Features

- Create, edit, and deactivate user accounts.
- Assign roles (Admin, Manager, Sales Executive) to users.
- Configure basic system settings and permissions.
- View system-wide reports across all teams.

5. High-Level System Architecture

The system follows a simple layered architecture. Each layer has one clear job, which makes the system easy to build, test, and explain. The diagram below shows the main components and how a call request flows through the system.



5.1 What Each Component Does

Component	Purpose
CRM Web Application (Frontend)	The screen the user sees in a browser. Shows leads, the Call button, call status, notes, and reports. Built as a web app so no separate software install is needed.
Backend / REST API	The "brain" of the system. Receives requests from the frontend (for example, "start a call"), checks if the user is allowed to do this, applies business rules, and talks to the database and the calling service.
Database	Stores all permanent data: users, leads, call records, notes, and follow-ups. This is the single source of truth for the CRM.
Calling Service	A third-party telephony/calling service that actually connects the Sales Executive's call to the customer's phone number and reports back the call status and duration. Explained further in Section 6.
Customer Phone	The end point of the call - the real phone number of the lead or customer being contacted.

6. Web Call Dialer Architecture

The Web Call Dialer is the part of the CRM that lets a Sales Executive make a phone call without leaving the browser. It does not depend on any single vendor. Instead, it is designed around a general "Calling Service", which is any third-party telephony/calling API that can connect two phone numbers and report call events back to our backend.

6.1 Why Use a Third-Party Calling Service

Building phone/telecom infrastructure from scratch (SIP trunks, carrier connections, call routing hardware) is complex and expensive. Instead, most CRMs integrate with an existing calling/telephony API provider over the internet. Our backend simply sends a request such as "call this customer number on behalf of this Sales Executive", and the calling service handles the actual telecom part.

6.2 Dialer Component Flow

```

CRM Frontend --(1) Click Call--> Backend API
Backend API --(2) Validate user & number--> Backend API
Backend API --(3) Request call--> Calling Service
Calling Service --(4) Connects--> Sales Executive & Customer
Calling Service --(5) Sends call events--> Backend API
Backend API --(6) Saves call record--> Database
Backend API --(7) Updates status--> CRM Frontend

```

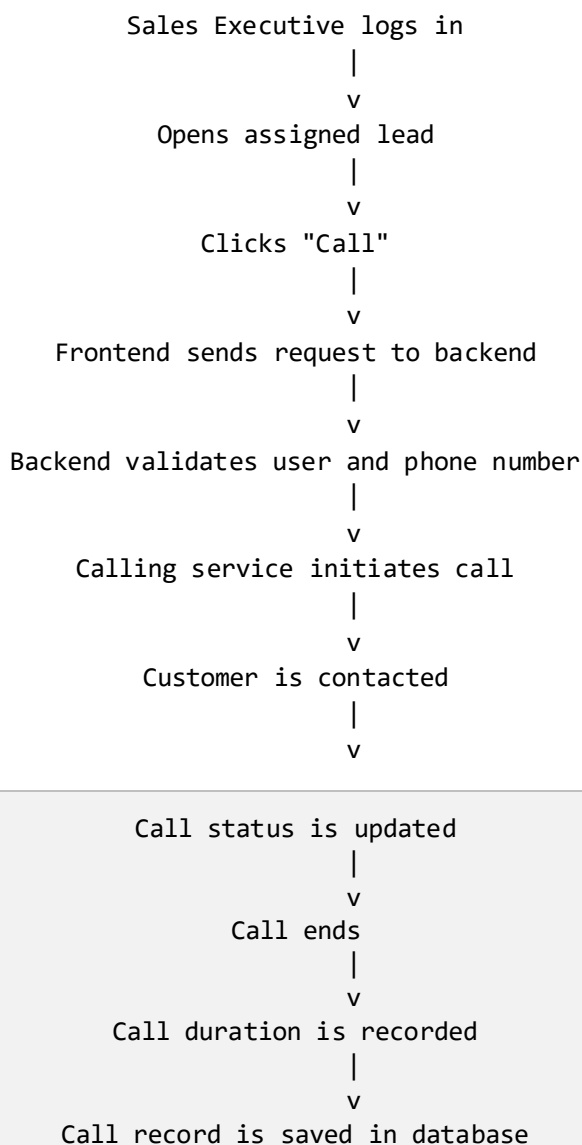
6.3 Responsibilities Split

Part	Responsibility
CRM Backend	Decides who is allowed to call, which lead is being called, and stores the final call result. It never talks to the telecom network directly.
Calling Service (Third-Party)	Handles the real telephone connection: dialing, ringing, connecting audio, detecting busy/no-answer, and measuring call duration. It sends status updates back to the CRM backend, usually through a callback/webhook.

This separation is important to explain in an interview: the CRM does not need to know about telecom protocols. It only needs to send a request to the calling service and listen for updates. This keeps the CRM simple and lets the calling service be replaced later without changing the rest of the system.

7. Complete Call Workflow

This section explains, step by step, what happens from the moment a Sales Executive logs in to the moment a call is fully recorded in the CRM.



|
v
CRM call history is updated

7.1 Step-by-Step Explanation

1. The Sales Executive logs in using their email/username and password. The backend checks the credentials and issues an access token.
2. The Sales Executive opens a lead that has been assigned to them and reviews the customer details.
3. The Sales Executive clicks the "Call" button next to the customer's phone number.
4. The frontend sends a request to the backend API, including the lead ID and the Sales Executive's ID.
5. The backend checks that the user is logged in, has permission to call this lead, and that the phone number is valid.
6. The backend sends a request to the calling service to start the call between the Sales Executive and the customer.
7. The calling service dials the customer's number. The call may be answered or it may fail (see below).
8. As the call progresses, the calling service sends status updates (Ringing, Connected, Completed, Failed, etc.) back to the backend.
9. When the call ends, the calling service reports the final status and the exact call duration.
10. The backend saves a new call record in the database with the lead ID, caller ID, status, duration, and timestamp.
11. The CRM screen updates automatically to show the new entry in the call history for that lead.

7.2 Successful Call

In a successful call, the customer answers, the conversation happens, and either party ends the call normally. The calling service reports status "Completed" along with the duration in seconds or minutes. The Sales Executive can then add notes (for example, "Customer asked for a callback next week") and update the lead status.

7.3 Failed Call

A call can fail for several reasons, for example the customer does not pick up (No Answer), the line is busy (Busy), the number is invalid or switched off (Failed/Invalid Number), or there is a network/technical issue at the calling service (Error). In all of these cases, the system still creates a call record with the correct failure status and zero or very short duration. This is important because it lets the Manager see how many attempts were made, even if the customer was never actually reached, and it lets the system prompt the Sales Executive to try again later or schedule a follow-up.

8. Call Status, Duration & Call History

Tracking call status, duration and history correctly is the core value of the Web Call Dialer feature. This section explains how each of these is handled.

8.1 Call Status Values

Status	Meaning
Initiated	The call request has been sent from the CRM to the calling service.
Ringing	The customer's phone is ringing.
Connected	The customer has answered and the call is in progress.
Completed	The call finished normally after being connected.
No Answer	The customer did not pick up the call.
Busy	The customer's line was busy.
Failed	The call could not be connected due to an invalid number or technical error.

8.2 Call Duration

Call duration is the total time the customer and the Sales Executive were actually connected and talking, measured in seconds. The calling service tracks the exact start time (when the customer answers) and end time (when either side hangs up) and reports the duration back to the CRM backend once the call ends. The CRM does not calculate this duration itself, since the calling service knows the exact connection time; the CRM simply stores the value it receives.

8.3 Call History

Every call, whether successful or failed, is saved as one record in the Calls table (see Section 9). Over time, this builds a complete call history for each lead. A Sales Executive can open a lead and see every past call, its status, duration, date and time, and any notes added after that call. A Manager can view this same history across their whole team to understand how leads are being followed up.

9. Simple Database / ER Diagram

The database is kept simple with four main tables. This is enough to support all the features described above, without adding unnecessary complexity.

9.1 Users Table

Field	Type	Description
user_id	Primary Key	Unique ID for the user.
name	Text	Full name of the user.
email	Text	Used for login, must be unique.
password_hash	Text	Encrypted password, never stored in plain text.
role	Text	Admin, Manager, or Sales Executive.
manager_id	Foreign Key	Links a Sales Executive to their Manager (nullable).
status	Text	Active or Inactive.

9.2 Leads / Customers Table

Field	Type	Description
lead_id	Primary Key	Unique ID for the lead/customer.
name	Text	Customer's name.
phone_number	Text	Customer's contact number.
email	Text	Customer's email (optional).
lead_status	Text	New, Contacted, Interested, Not Interested, Converted.
assigned_to	Foreign Key	user_id of the Sales Executive handling this lead.
created_at	Date/Time	When the lead was added to the system.

9.3 Calls Table

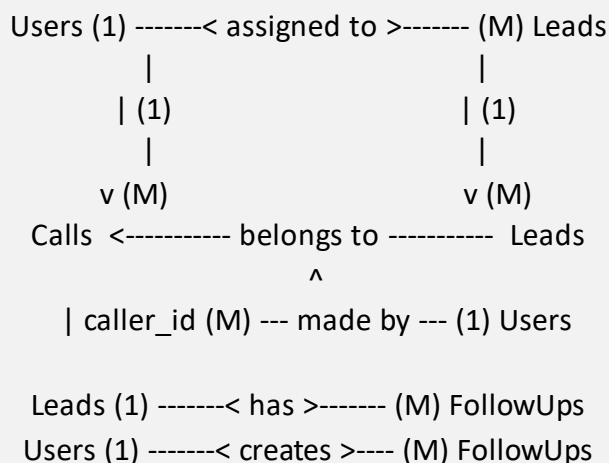
Field	Type	Description
call_id	Primary Key	Unique ID for the call record.

Field	Type	Description
lead_id	Foreign Key	The lead/customer that was called.
caller_id	Foreign Key	user_id of the Sales Executive who made the call.
call_status	Text	Completed, No Answer, Busy, Failed, etc.
duration_seconds	Number	Length of the connected call, in seconds.
call_time	Date/Time	When the call was made.
notes	Text	Notes added by the Sales Executive after the call.

9.4 FollowUps Table

Field	Type	Description
followup_id	Primary Key	Unique ID for the follow-up entry.
lead_id	Foreign Key	The lead this follow-up is for.
created_by	Foreign Key	user_id of the Sales Executive who set the follow-up.
followup_date	Date/Time	When the follow-up is due.
remark	Text	Short note on what to do at follow-up (for example, "Call back with pricing").
is_completed	Boolean	Whether the follow-up has been done.

9.5 Entity Relationship (ER) Diagram



In simple words: one User (Sales Executive) can have many Leads assigned to them. One Lead can have many Calls and many FollowUps over time. Every Call belongs to exactly one Lead and is made by exactly one User. This is a standard one-to-many relationship pattern, which is easy to explain and easy to build.

10. Basic API Design

The backend exposes a REST API. REST means the frontend talks to the backend using standard HTTP methods (GET, POST, PUT) over URLs that represent resources, such as /leads or /calls. Below are the main endpoints needed for this system.

Method	Endpoint	Purpose
POST	/api/auth/login	Log in a user and return an access token.
GET	/api/leads	Get the list of leads (filtered by role - own leads for Sales Executive, team leads for Manager, all for Admin).
GET	/api/leads/{id}	Get full details of a single lead, including its call history.
PUT	/api/leads/{id}	Update lead status or details.
POST	/api/calls/start	Start a new call for a given lead (this is what the "Call" button triggers).
POST	/api/calls/status-update	Receives call status/duration updates from the calling service (webhook).
GET	/api/calls/history/{leadId}	Get the full call history for a specific lead.

Method	Endpoint	Purpose
POST	/api/followups	Create a new follow-up for a lead.
GET	/api/followups/pending	Get pending follow-ups for the logged-in user or team.
GET	/api/reports/team	Get team-level call and lead reports (Manager/Admin).
POST	/api/users	Create a new user account (Admin only).
PUT	/api/users/{id}/role	Change a user's role or status (Admin only).

Each endpoint checks the user's role and access rights before performing the action. For example, only an Admin can successfully call /api/users to create a new account, even if another user tries to send the same request.

11. Authentication & Role-Based Access Control

11.1 Authentication

Authentication is the process of confirming who the user is. When a user logs in with their email and password, the backend checks the password against the stored (encrypted) password hash. If it matches, the backend generates a JWT (JSON Web Token) and sends it back to the frontend. This token is then sent with every future request, in the request header, so the backend knows which user is making the request without asking for the password again.

11.2 Role-Based Access Control (RBAC)

Role-Based Access Control means that what a user can see and do depends on their assigned role. The backend checks the role stored inside the JWT (or fetched from the database) before allowing any action.

Action	Admin	Manager	Sales Executive
View all leads (all teams)	Yes	No	No
View team leads	Yes	Yes	No
View own assigned leads	Yes	Yes	Yes
Make a call	No (not typical)	No (not typical)	Yes

Action	Admin	Manager	Sales Executive
Add notes / update lead status	Yes	Yes	Yes (own leads only)
View team reports	Yes	Yes	No
Create / manage user accounts	Yes	No	No

This table can be adjusted based on company policy, but the core idea for an interview is simple: every API request carries the user's identity and role, and the backend rejects any action that role is not allowed to perform, even if the request is technically valid.

12. Technology Stack and Why Each Technology is Used

Layer	Proposed Technology	Why This Technology
Frontend	React.js	A widely used JavaScript library for building interactive web pages. It makes it easy to update the screen in real time, for example showing live call status, without reloading the page.
Backend	Node.js + Express.js	Node.js allows JavaScript to run on the server, so the same language can be used on both frontend and backend. Express.js is a lightweight framework that makes it simple to build REST APIs.
Database	PostgreSQL	A reliable, open-source relational database. It fits well with structured data like Users, Leads, Calls, and FollowUps, and supports relationships between tables cleanly.
API Style	REST API	A simple, well-known standard for communication between frontend and backend using HTTP methods. Easy to test and easy to document.
Authentication	JWT (JSON Web Token)	A compact, secure way to confirm user identity on every request, without storing session data on the server for every user.
Calling / Telephony	Third-Party Telephony API	Building telecom infrastructure from scratch is unrealistic for most companies. A third-party

Layer	Proposed Technology	Why This Technology
		calling API handles the actual phone connection and reports call events back to the CRM.
Deployment	Cloud Platform (e.g., AWS / Azure / GCP)	Cloud hosting gives easy access to servers, storage, and scaling options, without needing to manage physical hardware.

This is intentionally a simple, standard stack. There is no need for advanced tools like microservices, Kubernetes, Kafka, or Redis at this stage - the system is not large enough to require them, and adding them would only make the design harder to explain and maintain.

13. Security

Since the system handles customer phone numbers and personal data, basic security practices are important.

- Passwords are never stored in plain text; they are stored as encrypted hashes.
- All communication between frontend, backend, and the calling service should use HTTPS, so data cannot be read while travelling over the network.
- Every API request is checked using the JWT token, so only logged-in, authorized users can access data.
- Role-Based Access Control (Section 11) prevents users from accessing data or actions outside their role.
- Input validation is done on the backend (for example, checking that a phone number is in a valid format) before any call is made or any data is saved.
- Sensitive actions, such as creating or deleting a user account, are restricted to the Admin role only.
- Call recordings or sensitive customer data, if stored, should follow the company's data privacy policy and applicable local regulations.

14. Basic Scalability

The system is designed to handle growth in a simple way, without over-engineering the solution.

- The backend API can be run on more than one server behind a basic load balancer, so more Sales Executives can use the system at the same time.
- The database can use indexes on commonly searched fields, such as `phone_number` and `lead_id`, to keep lookups fast as data grows.

- The calling service is external and already built to handle many simultaneous calls, so the CRM does not need to manage telecom scaling itself.
- As the number of users grows, the cloud platform allows adding more server capacity when needed, instead of buying new physical hardware.

At this stage, there is no need for complex tools like message queues, container orchestration, or caching layers. These can be added later if the system grows much larger, but they are not required for the current scope.

15. End-to-End Example

To make the whole system easy to explain, here is a simple, realistic walkthrough of one complete scenario.

1. Priya, a Sales Executive, logs in to the CRM using her email and password. The backend verifies her credentials and issues a JWT token.
2. Priya's dashboard shows 12 leads assigned to her. She opens the lead for "Rahul Sharma", a customer who enquired about a product last week.
3. Priya clicks the "Call" button next to Rahul's phone number.
4. The frontend sends a request to `POST /api/calls/start` with `lead_id` = Rahul's lead ID and `caller_id` = Priya's user ID.
5. The backend checks that Priya is logged in and that this lead is actually assigned to her. Both checks pass.
6. The backend sends a request to the calling service to connect Priya's line with Rahul's phone number.
7. The calling service dials the number. Rahul answers the call. The status changes from Initiated to Ringing to Connected.
8. Priya and Rahul talk for 3 minutes and 20 seconds. Rahul says he is interested and asks to be called back next Monday.
9. The call ends. The calling service reports `status` = Completed and `duration_seconds` = 200 to the backend.
10. The backend saves this as a new row in the Calls table, linked to Rahul's `lead_id` and Priya's `caller_id`.
11. Priya adds a note: "Interested, wants pricing details. Call back Monday." She updates the `lead_status` to "Interested" and creates a FollowUp entry for next Monday.
12. Priya's Manager, Arjun, opens his team dashboard the same evening and sees that Priya made 9 calls today, with an average call duration of 2 minutes 45 seconds, and 1 pending follow-up for Monday.
13. The Admin, reviewing the system weekly, sees that the whole sales team made 140 calls this week and that all user accounts and roles are correctly configured.

This one example touches almost every part of the system: login, role check, the Call button, the calling service, call status, call duration, notes, lead status, follow-ups, and manager/admin visibility.

16. Conclusion

The CRM-Based Web Call Dialer is a practical solution to a common problem in sales teams: calling and CRM data being disconnected from each other. By adding a simple "Call" button inside the CRM and connecting it

to a third-party calling service, Sales Executives can contact customers faster, and every call is automatically logged with its status and duration.

The system is kept intentionally simple: three clear roles (Admin, Manager, Sales Executive), a standard layered architecture (Frontend, Backend, Database, Calling Service), a small and focused database (Users, Leads, Calls, FollowUps), and a realistic, beginner-friendly technology stack. This makes the system easy to build, easy to maintain, and easy to explain in an interview, while still covering all the real requirements of a sales team's daily calling workflow.

